Retrieval-Augmented Generation

Grounding LLMs in Your Data

Day 3 - Session 2

KavinSchool

The Hallucination Problem

Why LLMs Make Things Up

KavinSchool

LLMs Hallucinate

LLMs generate text that **sounds correct** but may be **factually wrong**:

```
User: What is the company vacation policy?
LLM:  "Employees get 15 days of paid vacation per year,
       plus 5 personal days."
```

Is that true? The LLM has **no idea**. It generated a plausible-sounding answer from its training data.

Hallucinations happen because:

- The model wasn't trained on your private data
- Knowledge cutoff: doesn't know about recent events
- The model optimizes for **fluency**, not **accuracy**

The RAG Solution

Retrieval-Augmented Generation (RAG) = retrieve relevant documents first, then generate a response grounded in those documents.

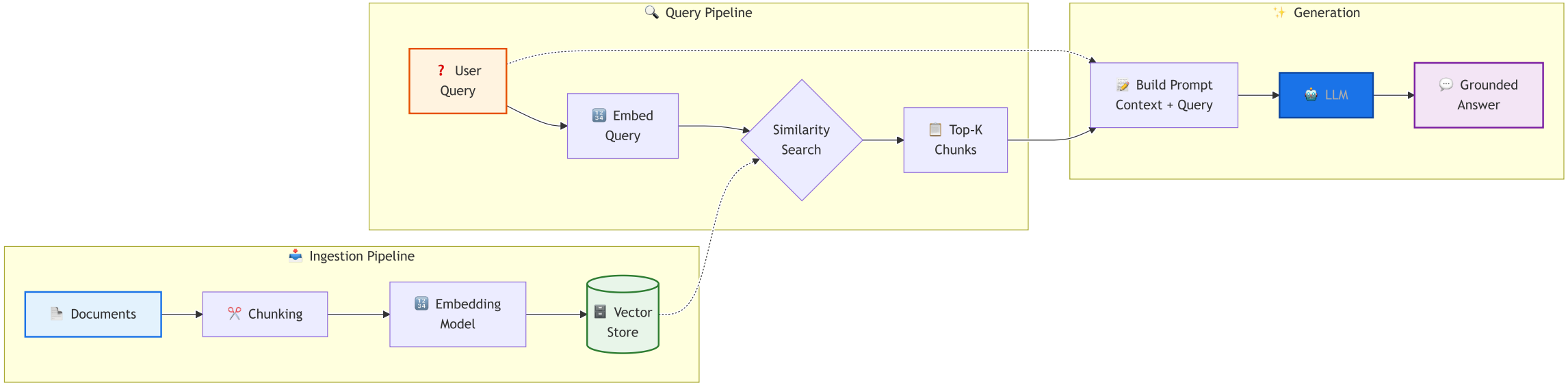
```
User: What is the company vacation policy?

RAG System:
1. Search company docs → finds "vacation-policy.md"
2. Inject relevant chunks into the prompt
3. LLM generates answer BASED ON the retrieved text

LLM: "According to the employee handbook (section 4.2),
      full-time employees receive 20 days of PTO per year."
```

> RAG doesn't eliminate hallucination entirely, but it dramatically reduces it by giving the model **real source material**.

RAG Architecture: The Big Picture



RAG: Two Pipelines

Pipeline	When It Runs	What It Does
Ingestion	Once (or on document updates)	Chunk docs → embed → store in vector DB
Query	Every user question	Embed query → search → retrieve → generate

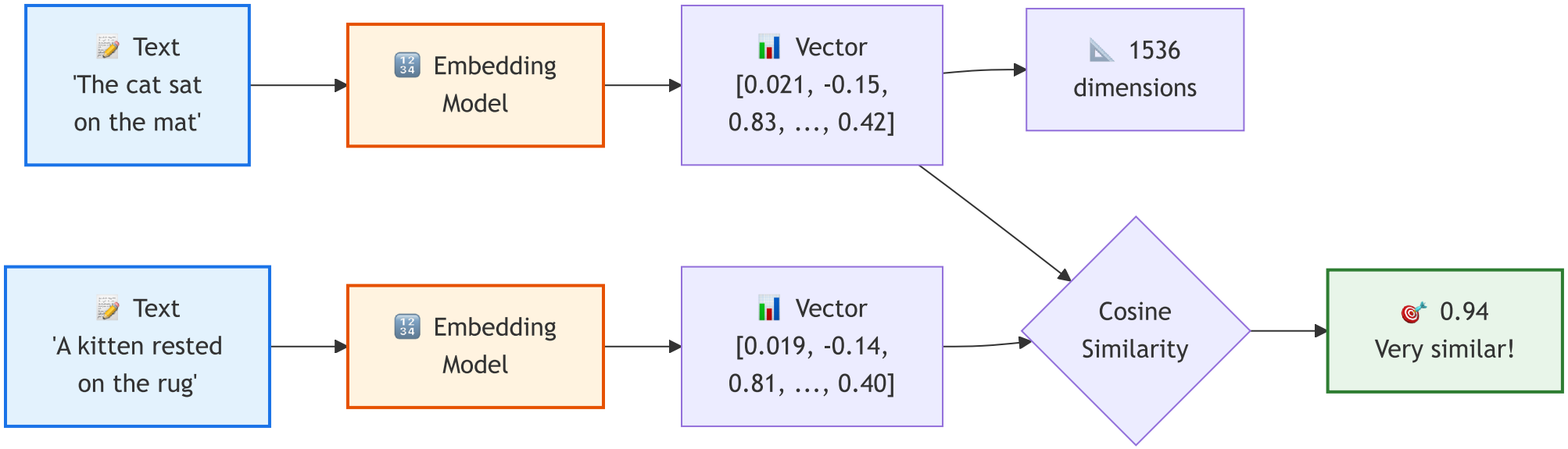
Embeddings

Turning Text Into Numbers

KavinSchool

What Are Embeddings?

An embedding is a **vector** (list of numbers) that represents the **meaning** of text.



> Similar meanings = similar vectors. "Happy" and "joyful" have vectors that are close together. "Happy" and "database" are far apart.

Embedding Models

Model	Provider	Dimensions	Cost (per 1M tokens)
`text-embedding-3-small`	OpenAI	1536	$0.02
`text-embedding-3-large`	OpenAI	3072	$0.13
`embedding-001`	Google	768	Free tier available
`voyage-3`	Voyage AI	1024	$0.06

Embedding Models

For this course: We'll use `text-embedding-3-small` via litellm (or the equivalent for your provider).

```
import litellm

response = litellm.embedding(
    model="text-embedding-3-small",
    input=["Hello, world!"]
)
vector = response.data[0]["embedding"]  # [0.021, -0.15, 0.83, ...]
print(f"Dimensions: {len(vector)}")     # 1536
```

Cosine Similarity

Cosine similarity measures how similar two vectors are (1.0 = identical, 0.0 = unrelated, -1.0 = opposite):

```
import numpy as np

def cosine_similarity(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

# Example
sim = cosine_similarity(embed("happy"), embed("joyful"))  # ~0.92
sim = cosine_similarity(embed("happy"), embed("database")) # ~0.15
```

Cosine Similarity

Score	Meaning	Example
0.90+	Very similar	"fix the bug" vs "resolve the issue"
0.70-0.89	Related	"Python error" vs "debugging code"
0.50-0.69	Somewhat related	"Python" vs "programming"
< 0.50	Unrelated	"Python" vs "chocolate cake"

Vector Databases

Storing and Searching Embeddings

KavinSchool

What Is a Vector Database?

A specialized database optimized for **storing vectors** and performing **fast similarity search**.

Feature	Traditional DB	Vector DB
Query type	Exact match (WHERE x = y)	Nearest neighbor (most similar)
Data	Rows and columns	High-dimensional vectors
Search	SQL queries	Cosine similarity / L2 distance
Use case	Structured data	Semantic search, RAG

Vector Database Options

Database	Type	Best For	Setup
FAISS	Library (local)	Prototypes, small datasets	`pip install faiss-cpu`
Chroma	Embedded DB	Dev/small production	`pip install chromadb`
Pinecone	Cloud service	Production at scale	API key required
Weaviate	Self-hosted/cloud	Enterprise	Docker or cloud
pgvector	PostgreSQL extension	Teams already on Postgres	Extension install

For this course: We'll use **FAISS** (fast, no server needed, included in requirements.txt).

What Is FAISS?

FAISS stands for **Facebook AI Similarity Search**, an open-source library from Meta AI for fast nearest-neighbor search over dense vectors.

Aspect	Detail
Built by	Meta AI (formerly Facebook AI Research)
Written in	C++ with Python bindings
Runs	In-process, no server (CPU or GPU)
Scales to	Millions to billions of vectors
Install	`pip install faiss-cpu` (or `faiss-gpu`)

What Is FAISS?

Why it fits this course:

- No external service or API key needed
- Fast exact and approximate search
- Index lives in memory, save/load to disk with one call

> FAISS handles the **search**, not the embeddings. You still create vectors with an embedding model, then add them to a FAISS index.

FAISS: Quick Start

```python
import faiss
import numpy as np

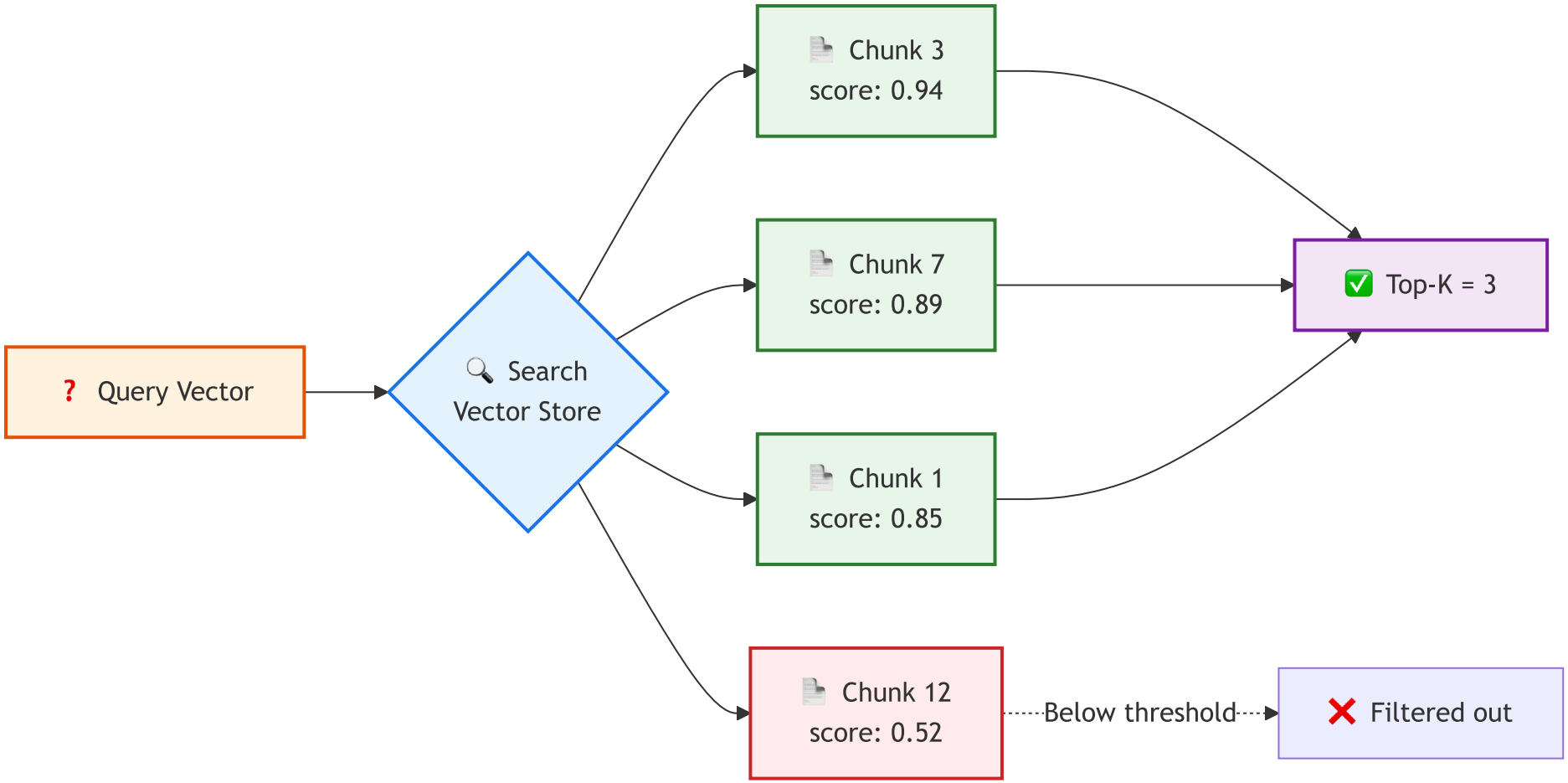
# Create an index for 1536-dimensional vectors
dimension = 1536
index = faiss.IndexFlatL2(dimension)

# Add vectors (numpy array, float32)
vectors = np.array(embeddings, dtype=np.float32)
index.add(vectors)

# Search: find 3 nearest neighbors
query_vector = np.array([query_embedding], dtype=np.float32)
distances, indices = index.search(query_vector, k=3)

# indices[0] = [7, 3, 12] — IDs of the 3 closest vectors
# distances[0] = [0.12, 0.34, 0.56] — L2 distances
```

Similarity Search: How It Works



Chunking

Preparing Documents for RAG

KavinSchool

Why Chunking Matters

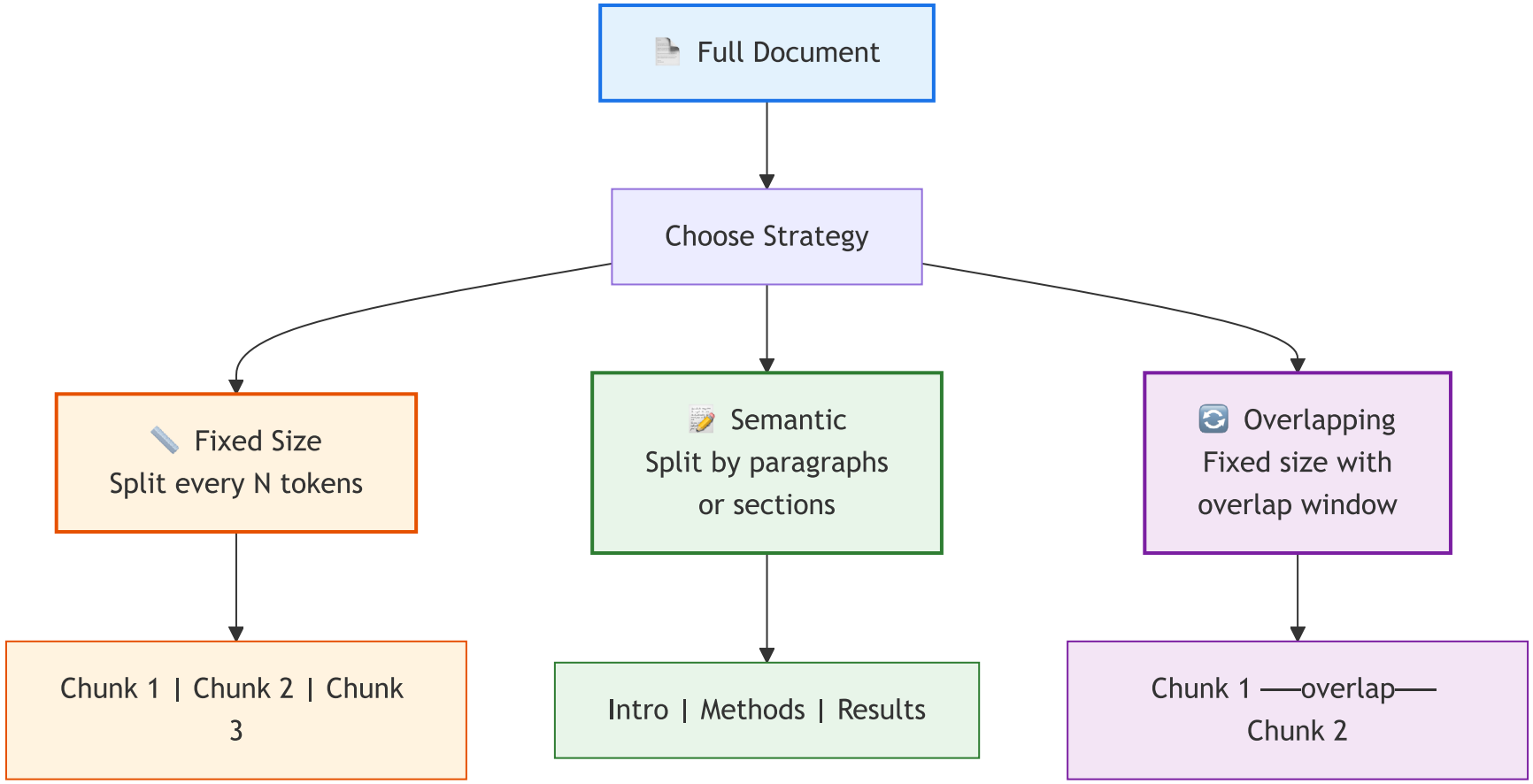
You can't embed an entire 50-page document as one vector. You need to **split it into chunks** first.

Why?

- Embedding models have **token limits** (8K for most)
- Smaller chunks = **more precise** retrieval
- Larger chunks = **more context** per result

> Chunk size is one of the most impactful RAG tuning parameters.

Chunking Strategies



Chunking in Practice

```python
def chunk_text(text, chunk_size=500, overlap=50):
    """Split text into overlapping chunks by character count."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunk = text[start:end]
        chunks.append(chunk)
        start = end - overlap  # overlap with previous chunk
    return chunks

# Better: split by paragraphs/sections first
def chunk_by_paragraphs(text, max_chunk_size=500):
    """Split by paragraph boundaries, merge small paragraphs."""
    paragraphs = text.split("\n\n")
    chunks, current = [], ""
    for para in paragraphs:
        if len(current) + len(para) > max_chunk_size and current:
            chunks.append(current.strip())
            current = ""
        current += para + "\n\n"
    if current.strip():
        chunks.append(current.strip())
    return chunks
```

Chunking Guidelines

Parameter	Recommendation	Why
Chunk size	200-500 tokens	Balances precision and context
Overlap	10-20% of chunk size	Prevents cutting mid-sentence
Split boundaries	Paragraphs, headers, sentences	Preserves meaning
Metadata	Source file, page number, section	Enables source citation

```
# Store chunks with metadata for source citation
chunks = [
    {"text": "...", "source": "handbook.md", "section": "Vacation Policy"},
    {"text": "...", "source": "handbook.md", "section": "Sick Leave"},
    {"text": "...", "source": "onboarding.md", "section": "First Week"},
]
```

Demo Time

Embeddings & Similarity Search

KavinSchool

Demo: Embeddings in Action

We'll embed 5 sentences, query with a related question, and visualize distance scores:

```
sentences = [
    "Python uses indentation to define code blocks.",
    "The capital of France is Paris.",
    "FAISS is a library for efficient similarity search.",
    "Machine learning models learn patterns from data.",
    "The quick brown fox jumps over the lazy dog.",
]

query = "How does Python structure its code?"
# Which sentence is the closest match?
```

> Open `day3/demos/demo-embeddings.py`

Session 2: Key Takeaways

1. **LLMs hallucinate.** RAG reduces this by grounding responses in real documents
2. **RAG has two pipelines.** Ingestion (chunk → embed → store) and Query (embed → search → generate)
3. **Embeddings capture meaning.** Similar text = similar vectors
4. **Vector databases enable fast search.** FAISS for prototypes, Pinecone/Chroma for production
5. **Chunking is critical.** Size, overlap, and boundaries all affect retrieval quality

> Next: Session 3, we build an end-to-end RAG chatbot!

Questions?

Session 2 Complete. Break Time!

KavinSchool